MediSlot: Project Documentation

1. Overview

1.1 Purpose

MediSlot is the backend and operations platform for an online doctor appointment service. Two kinds of users, doctors and patients, use one Flask web application. Every application action goes through Amazon API Gateway (HTTP API) and AWS Lambda functions. The project exists to show correctness under load: a slot is never booked beyond its capacity, a retried request never creates a second appointment, a reschedule always leaves the patient with exactly one appointment, and a freed seat is offered to exactly one waitlisted patient at a time.

1.2 Roles and portals

Role	Portal	Main capabilities
Doctor	/doctor/*	Register, log in, manage profile and photo, define consultation types, publish weekly availability and blackout dates, view day and week schedule with booked patients, mark appointment outcome (COMPLETED or NO_SHOW), cancel a slot with full refunds
Patient	/patient/*	Register, log in, manage profile, search doctors, browse slots, book, reschedule, cancel with a tiered refund, join and leave waitlists, accept hold offers, view appointments and refund history

Both portals are served by one Flask application on one EC2 t2.micro instance. A landing page offers Login as Doctor and Login as Patient. The application is a pure API client: it holds no database credentials and makes no database connection.

1.3 System guarantees and how each is enforced

Guarantee	Mechanism
A slot is never booked beyond its capacity	DynamoDB conditional transaction on the slot item using a version attribute and a capacity check (Section 5.2)
A retried booking never creates two appointments	Client Idempotency-Key stored in DynamoDB with a request hash and the stored response (Section 5.3)
A reschedule leaves exactly one appointment	Step Functions saga: claim new seat, move appointment, release old seat last, with compensation (Section 5.5)

Guarantee	Mechanism
A freed seat goes to one waitlisted patient at a time	One DynamoDB transaction moves a seat from available to held and creates the offer (Section 5.6)
Refunds are calculated consistently	Tier windows and percentages read from SSM; one ledger row per appointment (Section 7)
Every appointment change notifies the right people	EventBridge events, a Notification Lambda, an exactly-once dispatch log, and SNS (Section 8)
Users only see and change their own data	Identity and role come only from the authorizer context, never from a body, query string or header (Section 3)

2. Architecture

2.1 Components by layer

The numbering matches the architecture diagrams.

#	Component	Responsibility
1	Browser	Doctor or patient uses the web UI over HTTP
2	Amazon EC2 t2.micro, Flask	Serves both portals, guards /doctor/* and /patient/* by role, keeps the JWT in a server-side session, calls API Gateway only
3	Amazon API Gateway (REST)	Public routes (register, login, refresh) and protected routes (everything else), request validation, per-method throttling, access logging
4	Lambda authorizer	Reads the JWT secret from SSM, verifies HS256 signature and expiry, checks the jti denylist in DynamoDB, validates the role, returns an IAM policy with userId and role in the context, cache TTL 0
4a	Auth Lambda	Register (bcrypt hash), login (JWT and refresh token), refresh (rotation), logout (refresh token deleted, jti denylisted)
5	Application Lambdas	Doctor and patient business functions
6	Data layer	RDS MySQL, DynamoDB, S3, SSM Parameter Store
7	EventBridge Scheduler and job Lambdas	Daily slot materialiser, and one-time schedules for offer expiry and reminders

#	Component	Responsibility
8	EventBridge bus, Notification Lambda, Waitlist Processor Lambda, SNS	Domain events, exactly-once notifications, waitlist hold offers
9	Step Functions	Reschedule saga (patient side) and slot-cancel map (doctor side)
10	CloudWatch, SNS alarm topic	Logs, metrics, dashboard, alarms, traces
11	VPC	Public subnet for EC2, private subnets for Lambda and RDS, VPC endpoints

2.2 Application Lambda functions

Function	Role	Responsibility
Doctor Profile	Doctor	Get and update profile, timezone, hospital
Consultation Types	Doctor	Create, update, list and delete types with fee, duration and capacity
Availability and Blackouts	Doctor	Weekly patterns in local time and blackout dates
Schedule	Doctor	Day and week view with booked patients
Outcome	Doctor	Mark an appointment COMPLETED or NO_SHOW
Slot Cancel	Doctor	Close a slot and start the cancellation workflow
Photo Upload	Doctor	Generate a pre-signed S3 upload URL
Patient Profile	Patient	Get, update and delete profile
Doctor Search and Slot Lookup	Patient	Search doctors, public profile, free seats per slot
Booking	Patient	Idempotent, concurrency-safe booking
Cancel and Refund	Patient	Cancel, compute the refund tier, write the ledger, free the seat
Reschedule Starter	Patient	Check limits and start the saga
Waitlist	Patient	Join, leave, list waitlists, accept an offer
Refund History	Patient	List the patient's refund ledger
Waitlist Processor	System	Turns SlotSeatFreed into a hold offer
Notification	System	Sends each notification once through SNS
Slot Materialiser	System	Daily rolling 30-day slot generation

Function	Role	Responsibility
Offer Expiry and Reminder	System	Invoked by one-time schedules

2.3 Request flow

1. The browser sends an HTTP request to the Flask app on EC2.
2. Flask calls API Gateway with the access token from the server-side session.
3. API Gateway (HTTP API) invokes the Lambda authorizer for protected routes. A bad token returns 401. A valid token for the wrong role returns 403.
4. On success the authorizer returns userId and role in the request context, and API Gateway invokes the business Lambda.
5. The Lambda reads the acting user only from that context, then reads or writes RDS and DynamoDB.
6. State changes publish events to EventBridge, which route to the Notification and Waitlist Processor Lambdas.

2.4 Network and security boundaries

Item	Design
VPC	One custom VPC, 10.0.0.0/16
Public subnet	10.0.1.0/24 holds only the EC2 Flask app. Security group: allow 443 (and 80 redirect) from the internet. No rule or route to RDS
Private subnets, Lambda	10.0.2.0/24 and 10.0.3.0/24 in two availability zones. Lambda security group: outbound to RDS on 3306 and to the VPC endpoints
Private subnets, RDS	10.0.4.0/24 and 10.0.5.0/24. A DB subnet group spans two zones, the instance itself is single-AZ, no public IP, no internet route. Security group: allow 3306 from the Lambda security group only
API Gateway	Managed service outside the VPC. It invokes Lambda
No NAT gateway	Gateway endpoints for DynamoDB and S3 (free). Interface endpoints for SSM, SNS, EventBridge and Step Functions
IAM	One execution role per function with specific actions and resource ARNs. No wildcard actions
Encryption	AWS-managed keys only (aws/ssm, aws/rds). No customer-managed KMS keys

2.5 Event topology

Producers publish events to a single Amazon EventBridge event bus (Section 8). EventBridge rules route matching event types to their consumer Lambda functions. Event delivery retries and failure handling are described in Section 8.4.

2.6 Technology stack

Python (Lambda, Flask), AWS CloudFormation (infrastructure), GitHub and GitHub Actions (source control and CI/CD), pytest (tests), bcrypt and PyJWT (authentication).

3. Authentication and authorization

Authentication is built from scratch. No managed identity service is used.

3.1 Tokens

Token	Format	Lifetime	Storage
Access token	JWT, HS256. Payload has exactly sub (userId), role (doctor or patient), iat, exp and jti	15 minutes (SSM)	Flask server-side session
Refresh token	Opaque random string (at least 32 bytes)	7 days (SSM)	Only its SHA-256 hash is stored in DynamoDB with a TTL

3.2 Flows

Register. The Auth Lambda enforces the password policy (at least 10 characters, one letter, one digit), hashes the password with bcrypt at a work factor of 12 or higher (SSM), and inserts the user. Email is unique across both roles, so a doctor and a patient cannot share an address. A duplicate returns 409.

Login. The Lambda finds the user, checks that the role matches the login page, and verifies the bcrypt hash. Any failure returns a generic 401 and increments the login-failure metric. Success returns a JWT and a refresh token.

Refresh. The Lambda hashes the presented token and looks it up. A conditional delete removes the old token, so it works once. A new JWT and a new refresh token are issued (rotation).

Logout. A protected route. The Lambda deletes the refresh token and writes the access token's jti to the denylist with a TTL equal to the token's remaining life. The next use of that token returns 401.

3.3 Authorizer

1. Read the signing secret from SSM at start.
2. Verify the signature, pinning the algorithm to HS256 so a token cannot downgrade it.

3. Check expiry.
4. Check that the jti is not on the denylist.
5. Check that the role is allowed for the route (doctor-only routes reject patient tokens with 403).
6. Return an IAM policy with userId and role in the context. Handlers read the acting user only from this context.
7. Cache TTL is 0. Any cache would let a logged-out token through, so every request is evaluated. This is the justification to state in the design review.

4. Data model

4.1 Why two stores

Store	Used for	Reason
RDS MySQL	Users, profiles, hospitals, specialisations, consultation types, availability, blackouts, appointments, refund ledger	Relational, joined, searched by many attributes, auditable. Needs foreign keys and composite indexes
DynamoDB	Slots, seats, waitlists, hold offers, idempotency keys, counters, tokens, denylist, dispatch log	The contended write target. Conditional writes and transactions give hard guarantees. TTL manages lifecycles. Access is by key

The appointment is the system of record in RDS. DynamoDB holds the live, contended state. The two are not updated atomically, and Section 5.7 states what that means.

4.2 RDS tables

Table	Key columns	Notes
users	id (PK), email (UNIQUE), password_hash, role, created_at	Email unique across both roles
hospitals	id, name, address, city	Index on (city) and (name)
specialisations	id, name (UNIQUE)	
doctors	user_id (PK, FK users), full_name, licence_number (UNIQUE), specialisation_id (FK), years_experience, qualifications, hospital_id (FK), timezone (IANA), photo_key	Indexes for search: (specialisation_id), (hospital_id), (full_name)
patients	user_id (PK, FK users), full_name, date_of_birth, gender, phone, city, emergency_contact_name, emergency_contact_phone, notes	Free-text notes only, no medical records

Table	Key columns	Notes
consultation_types	id, doctor_id (FK), name, duration_minutes, fee_amount, capacity	Quick 15, Standard 30, Detailed 60 with capacity 1. Health Camp 60 with capacity 2 to 10 (CHECK constraint)
availability_patterns	id, doctor_id (FK), day_of_week, start_time, end_time, consultation_type_id (FK)	Times are local to the doctor
blackout_dates	id, doctor_id (FK), blackout_date, reason	UNIQUE (doctor_id, blackout_date)
appointments	id, patient_id, doctor_id, slot_id, consultation_type_id, slot_start_utc, slot_end_utc, status, fee_amount, reschedule_count, version, cancelled_by, created_at, updated_at	Status: CONFIRMED, COMPLETED, CANCELLED_BY_PATIENT, CANCELLED_BY_DOCTOR, NO_SHOW. Indexes: (patient_id, slot_start_utc), (doctor_id, slot_start_utc), (slot_id). A generated column that is the patient and slot pair for CONFIRMED rows only, UNIQUE, stops a second active appointment in one slot
refund_ledger	id, appointment_id (UNIQUE), patient_id, base_amount, refund_percent, refund_amount, tier, reason, created_at	One row per appointment, so a retry cannot refund twice

4.3 DynamoDB tables

Table	Partition key	Sort key	Indexes	Purpose and access patterns
Slots	slotId	none	GSI doctor-date: doctorId, startUtc	Slot item with capacity, bookedCount, heldCount, version, status, tz, startUtc, endUtc. Point lookup by slot, range by doctor and date
Seats	slotId	patientId	none	One item per patient per slot. A conditional put (item must not exist)

Table	Partition key	Sort key	Indexes	Purpose and access patterns
				enforces one seat per patient per slot
Waitlist	slotId	joinedAt#patientId	GSI patient: patientId, slotId	Ordered by join time. First waiting patient is the first item in the partition
HoldOffers	slotId	offerId	GSI patient: patientId, expiresAt	status LIVE, ACCEPTED, EXPIRED, CANCELLED. TTL for cleanup
Idempotency	key (patientId#Idempotency-Key)	none	none	status, requestHash, stored response. TTL 24 hours
Counters	counterKey	none	none	PATIENT_DOCTOR#patient#doctor (limit 3) and PATIENT_WAITLIST#patient (limit 5). Conditional increments
Tokens	pk (RT#hash or DENY#jti)	none	none	Refresh tokens (TTL 7 days) and the logout denylist (TTL equal to remaining token life)

4.4 SSM parameters

All parameters live under /medislot/{env}/.

Parameter	Default	Used by
jwt/signing-secret	SecureString	Auth Lambda, authorizer
jwt/access-ttl-seconds	900	Auth Lambda
jwt/refresh-ttl-seconds	604800	Auth Lambda
auth/bcrypt-cost	12	Auth Lambda
refund/tier-hours and tier-percents	48 to 100, 24 to 75, 6 to 50, below 6 to 0	Cancel Lambda

Parameter	Default	Used by
refund/doctor-cancel-percent	100	Slot-cancel workflow
booking/window-days	30	Materialiser
booking/max-upcoming-per-doctor	3	Booking Lambda
booking/retry-max-attempts and backoff-base-ms	5 and 20	Booking Lambda
idempotency/ttl-hours	24	Booking Lambda
reschedule/max-count and cutoff-hours	2 and 6	Reschedule Starter
waitlist/hold-minutes	15	Waitlist Processor
waitlist/max-per-patient	5	Waitlist Lambda
reminder/offsets-hours	24 and 2	Booking Lambda
test/fail-release	false	Reschedule saga (forces Test 3)

5. Concurrency design note

5.1 The contended resource

The slot item in the Slots table is the contended resource. It carries capacity, bookedCount, heldCount, version and status. A seat is available when status is OPEN and bookedCount plus heldCount is less than capacity.

5.2 Booking with optimistic concurrency

1. Read the slot: Perform a strongly consistent read of the slot item from DynamoDB. Record its current version, status, bookedCount, heldCount, and capacity.
2. Execute a transaction: Submit one DynamoDB TransactWriteItems operation containing:
    - Update the slot: Proceed only if status = OPEN, version = :v, and bookedCount + heldCount < capacity. Increment bookedCount and version by one.
    - Create the seat: Insert a seat item for the patient and slot only if that item does not already exist.
3. Handle the result: If the transaction succeeds, the booking is confirmed. If it fails, inspect the cancellation reasons where available:
    - Existing seat: return HTTP 409 Conflict (already booked).
    - Slot full or closed: return HTTP 409 Conflict.
    - Version mismatch or concurrent modification: proceed to the retry step.

4. Retry conflicts: Re-read the slot, wait using exponential backoff with jitter, and retry the transaction. Limit attempts to the configured maximum of five, retrieved from AWS Systems Manager Parameter Store. Increment the BookingConflictRetries metric for each retry.
5. Handle exhausted retries: If the maximum number of attempts is reached, return HTTP 409 Conflict with a retry-later message.
6. Concurrency guarantee: No explicit lock or single-threaded queue is required. DynamoDB conditional writes and transactions ensure that concurrent successful bookings cannot exceed the slot's capacity, provided all booking paths use this transaction logic.
7. Example: For a slot with capacity 3 and 20 simultaneous booking requests, at most three bookings can succeed if the slot initially has three available seats. The remaining requests cannot book once capacity is exhausted. Health Camp slots use the same logic, with capacities ranging from 2 to 10.

5.3 Idempotency

The key is patientId plus the Idempotency-Key header, so one patient's key cannot collide with another's. A missing header returns 400.

- First request: conditional put of an IN_PROGRESS record with the request hash and a 24-hour TTL.
- Replay with the same hash and a COMPLETED record: return the stored status code and body, so the original appointment comes back and no second one is created.
- Replay while IN_PROGRESS: return 409 with a retry-later message.
- Same key with a different request hash: return 422.
- Final outcomes, including deterministic 4xx and 409 results, are stored so a replay gets the same answer. After a 5xx (for example an RDS failure) the key is deleted so the client can retry cleanly.

5.4 Per-doctor limit and one seat per slot

The limit of 3 upcoming appointments per doctor is a conditional counter in the Counters table, incremented before the seat claim and undone if the claim fails. It is decremented on cancel. When the counter is at the limit, the Lambda checks it lazily against RDS (CONFIRMED appointments whose slot_start_utc is in the future) and corrects it if it is stale. This matters because appointments are never closed automatically. The seat item (slotId plus patientId) enforces one seat per patient per slot. A booking for a slot whose start time is in the past is rejected before any write.

5.5 Reschedule saga

The Reschedule Starter Lambda checks that the appointment is CONFIRMED and owned by the caller, that reschedule_count is below 2, that the start is more than 6 hours away, and that the target is a different slot in the future. It writes a conditional marker on the appointment version so two sagas cannot run at once, then starts the Step Functions execution (execution name = appointmentId plus reschedule count, so a repeat start is rejected).

Step	Action	If it fails
1	Claim a seat in the new slot (same conditional transaction as booking)	New slot full: fail with 409. The original stays CONFIRMED and nothing needs undoing
2	Update the appointment in RDS (new slot, reschedule_count + 1, conditional on version)	Compensation A: release the new seat. The original stays CONFIRMED
3	Release the seat in the old slot, with retries and backoff	Compensation B: restore the RDS row to the original slot and release the new seat. The original stays CONFIRMED
4	Publish AppointmentRescheduled and SlotSeatFreed (old slot), replace the reminder schedules, notify patient and doctor	Retried. The saga has already succeeded

The release is last because it cannot be undone safely: once the old seat is free, another patient can take it. A reschedule moves the appointment. It is not a cancel, so there is no refund and no fee. Test 3 sets the SSM flag test/fail-release so step 3 fails on demand, and the test proves the patient still holds exactly one appointment. The ReschedulesCompensated metric counts compensations, and any compensation raises an alarm.

5.6 Waitlist exclusivity

Join. A conditional counter allows at most 5 waitlists per patient. The entry is stored with sort key joinedAt#patientId, so the partition is ordered by join time. **Leave.** The entry is deleted and the counter decremented. If the patient holds a live offer, the offer is cancelled, the held seat is released and SlotSeatFreed is published.

Issuing an offer. On SlotSeatFreed the Waitlist Processor reads the first WAITING entry and runs one transaction:

- Update the slot: condition version = v AND bookedCount + heldCount < capacity, then heldCount + 1 and version + 1.
- Update the waitlist entry: condition status = WAITING, set status OFFERED.
- Put the offer: status LIVE, expiresAt = now + 15 minutes (SSM).

If any condition fails the whole transaction fails: either no free seat is left or another handler already took that patient. So one freed seat becomes exactly one offer. While held, heldCount counts against capacity, so normal booking cannot take the seat. After the transaction the handler creates a one-time schedule for expiresAt and publishes WaitlistOfferIssued.

Accepting. One transaction moves the offer from LIVE to ACCEPTED (condition: status is LIVE and expiresAt is later than now), turns heldCount into bookedCount and writes the seat item. Then the appointment is written to RDS. If the RDS write fails, the held seat is released, the offer is marked FAILED and SlotSeatFreed is published again.

Expiring. When the one-time schedule fires, a conditional update moves the offer from LIVE to EXPIRED, the held seat is released and the entry is removed. WaitlistOfferExpired and SlotSeatFreed are published, so the next patient in line gets the seat. A second run finds the offer already EXPIRED and does nothing. As a safeguard, Booking and the Waitlist Processor also treat a LIVE offer past its expiresAt as expired and apply the same conditional update, so a missed schedule cannot hold a seat forever.

A slot cancelled by the doctor has status CANCELLED, so the Waitlist Processor ignores it. Its waitlist entries and live offers are cleared.

5.7 What is guaranteed and what is not

Guaranteed	How
No overbooking	Conditional transaction on the slot item
One seat per patient per slot	Seat item that must not already exist
One appointment per idempotency key	Stored key, request hash and response
Exactly one appointment after a reschedule, even on failure	Saga with compensation and a last, irreversible step
One live offer per freed seat	Single transaction for seat, entry and offer
Notification at most once per event and recipient	Conditional put in the dispatch log

Known window	Mitigation	Residual risk
DynamoDB and RDS are not updated atomically. A Lambda that dies after the seat claim and before the RDS write leaves a PENDING seat	The seat is released if the RDS write fails. Seat items carry claimedAt, and Slot Lookup and Booking release PENDING seats older than 2 minutes that have no RDS appointment (lazy reclaim)	The seat is held until the next lookup or booking touches the slot
A Lambda that dies after the RDS commit and before the event is published loses that event	Publication retries with backoff before the handler returns	The notification for that one appointment may be missed
A notification is lost if SNS publish fails after the dispatch-log write	The log entry is deleted before the retry	A crash between the two steps loses the message

6 Materialisation

1. For each date in the rolling 30-day window, skip dates in the doctor's blackout_dates.
2. For each pattern on that weekday, step through the local range from the pattern start in multiples of the consultation duration, keeping only slots that end on or before the pattern end.
3. Build each slot as a local datetime with the doctor's timezone (Python zoneinfo), then convert it to UTC. Because the arithmetic is done in local time, 09:00 local stays 09:00 local on every date, even when the UTC offset changes.
4. Write each slot with a conditional put that requires the slotId not to exist. Re-running the job for a processed date creates no duplicates.

7. Refunds

7.1 Tiers

Refund is calculated from the difference between the appointment start and the cancellation time, both in UTC. Windows and percentages come from SSM.

Cancellation timing	Refund	Deduction
More than 48 hours before	100%	0%
24 hours up to and including 48 hours before	75%	25%
6 hours up to but not including 24 hours before	50%	50%
Less than 6 hours before	0%	100%
Cancelled by the doctor	100%	0%, regardless of timing
Marked NO_SHOW	0%	100%

7.2 Boundary instants

The brief does not say which tier an exact boundary belongs to. This design keeps each lower bound inclusive.

Time before start	Refund
48:00:01	100%
48:00:00	75%
24:00:00	75%
23:59:59	50%
6:00:00	50%

Time before start	Refund
5:59:59	0%

7.3 Ledger

The refund_ledger table gets one row per appointment (UNIQUE appointment_id) with the base amount, percentage, refund amount (rounded half up to 2 decimals), tier and reason. Nothing is charged. Patients read their ledger through the refund history endpoint. A conditional status change (CONFIRMED to CANCELLED) plus the unique ledger key means a double click or retry cannot refund twice.

8. Events and notifications

8.1 Event catalogue

Every event uses one envelope: eventId, eventType, occurredAt, correlationId and detail.

Event	Published by	Consumers
AppointmentBooked	Booking, Offer accept	Notification
AppointmentRescheduled	Reschedule saga	Notification
AppointmentCancelled (cancelledBy patient or doctor)	Cancel, Slot-cancel workflow	Notification
SlotCancelled	Slot-cancel workflow	Notification
SlotSeatFreed	Cancel, Reschedule saga, Offer expiry	Waitlist Processor
WaitlistOfferIssued	Waitlist Processor	Notification
WaitlistOfferExpired	Offer expiry	Notification
AppointmentCompleted	Outcome	Notification

A doctor cancelling a whole slot publishes SlotCancelled and AppointmentCancelled but not SlotSeatFreed, because the slot is closed and no seat should be offered.

8.2 Notification matrix

Event	Recipient
Appointment booked	Patient and doctor
Appointment rescheduled	Patient and doctor
Appointment cancelled by patient	Patient and doctor
Appointment cancelled by doctor	Patient only

Event	Recipient
Waitlist hold offer issued	Patient
Waitlist hold offer expired	Patient
Reminder 24 hours before	Patient
Reminder 2 hours before	Patient

8.4 Failure handling without queues

Layer	Behaviour
EventBridge rule target	Retry policy: up to 8 attempts within 1 hour
Lambda asynchronous invocation	2 retries, then an on-failure destination that publishes to the alarm SNS topic
Handlers	Idempotent, so every retry is safe
Alarms	Dropped asynchronous events above zero, on-failure deliveries above zero, Lambda error rate (replaces the DLQ-depth alarm)

9. Scheduled jobs

Job	Trigger	Idempotency guard
Slot materialiser	Recurring, daily	Conditional put per slotId
Offer expiry	One-time schedule at the offer's expiresAt	Conditional update LIVE to EXPIRED
Reminders (24 hours and 2 hours)	Two one-time schedules per appointment	Dispatch-log conditional put, plus a check that the appointment is still CONFIRMED and the slot time is unchanged

10. API documentation

10.1 Conventions

Item	Rule
Base URL	https://{api-id}.execute-api.{region}.amazonaws.com/{stage}
Authentication	Authorization: Bearer {accessToken} on every route except register, login and refresh

Status	Meaning
400	Validation failed (including password policy, missing Idempotency-Key, slot in the past)
401	Missing, expired, tampered, wrongly signed or logged-out token; bad credentials
403	Valid token for the wrong role
404	Not found, or not owned by the caller (existence is not revealed)
409	Conflict: slot full, duplicate email, state not allowed, key still in progress
422	Rule violation: limit reached, idempotency key reused with a different request
429	Throttled
500	Server error

10.2 Authentication (both roles)

Method and path	Auth	Request	Success	Errors
POST /auth/doctors/register	Public	email, password, fullName, licenceNumber, specialisation, hospitalName, hospitalAddress, city, yearsExperience, qualifications, languages, timezone,	201 userId	400 policy or validation, 409 email exists
POST /auth/patients/register	Public	email, password, fullName, dateOfBirth, gender, phone, city, emergencyContact	201 userId	400, 409
POST /auth/doctors/login	Public	email, password	200 accessToken, refreshToken, expiresIn, role	401 generic
POST /auth/patients/login	Public	email, password	200 as above	401 generic
POST /auth/refresh	Public (valid refresh token)	refreshToken	200 new accessToken and refreshToken	401

Method and path	Auth	Request	Success	Errors
POST /auth/logout	Protected, both roles	refreshToken	204	401

Password policy: at least 10 characters with at least one letter and one digit. A login for the wrong role returns the same generic 401.

10.3 Doctor APIs (doctor role; patient tokens get 403)

Method and path	Request	Success	Errors
GET /doctors/me	none	200 profile	
PUT /doctors/me	Profile fields to change	200 profile	400
POST /doctors/me/photo	contentType	200 uploadUrl (pre-signed PUT, short expiry) and photoKey	400
POST /doctors/me/consultation-types	name, durationMinutes, feeAmount, capacity	201 type	400 (Quick 15, Standard 30, Detailed 60 with capacity 1; Health Camp 60 with capacity 2 to 10)
GET /doctors/me/consultation-types	none	200 list	
PUT /doctors/me/consultation-types/{id}	Fields to change	200 type	400, 404
DELETE /doctors/me/consultation-types/{id}	none	204	404, 409 if future slots use it (design choice)
PUT /doctors/me/availability	patterns: dayOfWeek, startTime, endTime, consultationTypeId (local time)	200 patterns	400 overlap or bad time
POST /doctors/me/blackouts	date, reason	201 blackout	400, 409 duplicate date

Method and path	Request	Success	Errors
GET /doctors/me/blackouts	none	200 list	
DELETE /doctors/me/blackouts/{id}	none	204	404
GET /doctors/me/schedule?date=	date (doctor local date)	200 slots with capacity, bookedCount and booked patients	400
GET /doctors/me/appointments?from=&to=&status=	filters, limit, cursor	200 page of appointments	400
DELETE /doctors/me/slots/{slotId}	none	202 with the number of affected appointments. Each patient is cancelled, fully refunded and notified	404, 409 already cancelled
POST /doctors/me/appointments/{id}/outcome	outcome: COMPLETED or NO_SHOW	200 appointment	404, 409 if not CONFIRMED or the slot has not started

10.4 Patient APIs (patient role; doctor tokens get 403)

Method and path	Request	Success	Errors
GET /patients/me	none	200 profile	
PUT /patients/me	Profile fields to change	200 profile	400
DELETE /patients/me	none	204	409 while upcoming appointments or live offers exist
GET /doctors?name=&hospital=&specialisation=&city=	filters, limit, cursor	200 page of doctors	400
GET /doctors/{doctorId}	none	200 public profile with consultation types and fees	404

Method and path	Request	Success	Errors
GET /doctors/{doctorId}/slots?from=&to=&consultationTypeId=	date range	200 slots with free seats (capacity minus booked minus held)	400
POST /appointments	Header Idempotency-Key. Body: slotId	201 appointment. A replay returns the original appointment	400 missing key or slot in the past, 409 slot full or key in progress, 422 limit of 3 upcoming with this doctor or key reused with a different request
GET /appointments?status=&from=&to=	filters, limit, cursor	200 the caller's appointments, upcoming and past	
GET /appointments/{id}	none	200 appointment	404
PATCH /appointments/{id}	targetSlotId	202 saga started. GET the appointment to see rescheduleStatus (PENDING, SUCCEEDED, COMPENSATED)	404, 409 target full or appointment not CONFIRMED, 422 more than 2 reschedules or within 6 hours of the start
DELETE /appointments/{id}	none	200 refundPercent, refundAmount, tier	404, 409 not cancellable
POST /slots/{slotId}/waitlist	none	201 entry	409 slot not full or already waiting, 422 already on 5 waitlists
DELETE /slots/{slotId}/waitlist	none	204	404
GET /patients/me/waitlists	none	200 entries with any live offer and its expiresAt	
POST /waitlist-offers/{offerId}/accept	none	201 appointment	404, 409 offer expired or not live
GET /patients/me/refunds	limit, cursor	200 refund ledger entries	

10.5 Appointment resource

id, status (CONFIRMED, COMPLETED, CANCELLED_BY_PATIENT, CANCELLED_BY_DOCTOR, NO_SHOW), slot (slotId, startUtc, endUtc, localStart, timezone),

doctor (id, name), consultationType (name, fee), rescheduleCount, rescheduleStatus, cancelledBy, createdAt.

10.6 Data access rules

Caller	May read or change	Otherwise
Patient	Own profile, own appointments, own waitlists, offers and refunds	404 for another patient's record
Doctor	Own profile, own schedule and own appointments	404 for another doctor's record
Any authenticated caller	Public doctor profile and slot availability (patients)	

10.7 Web application routes

Route	Behaviour
/	Landing page with Login as Doctor and Login as Patient
/doctor/login and /patient/login	Login form with a link to register. On success the app calls the platform login API and stores the JWT in a server-side session
/doctor/*	Doctor portal. A patient session is redirected to the landing page
/patient/*	Patient portal. A doctor session is redirected to the landing page
Logout	Clears the session and calls POST /auth/logout

The web app holds no database credentials. Every read and write goes through API Gateway.

11. Observability

11.1 Custom metrics

Namespace: medislot. Metrics are emitted from Lambda logs in CloudWatch embedded metric format, so VPC-attached functions need no extra endpoint.

Metric	Meaning
AppointmentsBooked	Successful bookings
BookingsRejectedFull	Bookings rejected because the slot was full
BookingConflictRetries	Retries after a version conflict (proves real contention)

Metric	Meaning
ReschedulesSucceeded and ReschedulesCompensated	Saga outcomes
CancellationsByTier	Cancellations per refund tier
RefundAmountTotal	Sum of refund amounts written to the ledger
WaitlistPromotions and OffersExpired	Offer lifecycle
SlotUtilisationPercent	Booked share of capacity
NotificationsDispatched	Messages sent through SNS

11.2 Dashboard

The medislot-operations dashboard is defined in the monitoring stack and never created by hand. It has widgets for API Gateway (request count, 4xx and 5xx rate, latency p50 and p99), Lambda (invocations, errors, duration p50 and p99, throttles per function), DynamoDB (consumed capacity, throttled requests, conditional check failures), Step Functions (started, failed, compensation entries), RDS (CPU, connections, free storage), authentication (login successes, login failures, token validation rejections) and asynchronous delivery (dropped events and failed destination deliveries, which replace the SQS widgets).

11.3 Alarms

All alarms deliver to the alarm SNS topic and are defined in the monitoring stack.

Alarm	Condition
Asynchronous delivery failure (replaces DLQ depth)	Dropped asynchronous events above zero, or any invocation of the on-failure destination
Saga compensation	Any compensation entry (ReschedulesCompensated above zero)
Conditional check failure rate	DynamoDB conditional check failures above a threshold
API 5xx rate	Above a threshold
RDS CPU	Above 70%
Lambda error rate	Above a threshold on any function

For the demo, trigger one alarm on purpose, for example by forcing a saga compensation with the test flag.

12. Infrastructure as code and delivery

12.1 Stacks

Stacks are parameterised and linked by cross-stack exports. Every resource carries the tags Project, Environment and Owner for cost allocation.

Stack	Contents
network	VPC, subnets, route tables, security groups, VPC endpoints
data	RDS, DynamoDB tables (TTL and point-in-time recovery), S3 buckets, non-secret SSM parameters
auth	Per-function IAM roles, Auth Lambda, Lambda authorizer
api	API Gateway, business Lambdas, Step Functions state machines
events	EventBridge bus and rules, SNS topics, Notification and Waitlist Processor Lambdas, scheduled-job Lambdas and the scheduler role
web	EC2 instance, instance profile, UserData that pulls the app from S3
monitoring	Dashboard and alarms

12.2 Repository layout

infra (templates), src/lambdas, src/web (Flask app), tests/unit, tests/integration, .github/workflows, docs. The repository includes a .gitignore from Milestone 1 and never holds credentials, .env files or build artefacts.

12.3 Branching and review

main is protected. All work happens on short-lived branches named feature/, fix/, infra/ or docs/ followed by a short description. Every change reaches main through a pull request that has a clear title and description (what, why, how tested, what to check), a green pipeline and at least one peer approval. Branch protection requires a pull request, one approving review, passing status checks, an up-to-date branch and dismissal of stale approvals, blocks force pushes and deletion, and applies to administrators.

13. Testing

13.1 Unit tests

Refund tier calculation at every boundary instant, slot materialisation, reschedule limit and cutoff enforcement, waitlist ordering, capacity arithmetic, password policy, JWT claim handling and idempotency key states.

13.2 Mandatory integration tests

Each one runs against deployed infrastructure and gates the pipeline.

Test	Setup	Action	Pass condition
1. Concurrency	Health Camp slot with capacity 3, 20 or more patients	Fire all booking requests at once	Exactly 3 succeed, the rest get 409, bookedCount equals 3, retries are visible in the metric
2. Idempotency	One slot, one patient	Send the same Idempotency-Key several times, including in parallel	Exactly one appointment, every response returns the same appointment
3. Reschedule saga	Confirmed appointment, target slot available, test/fail-release set	Reschedule	The saga compensates, the patient holds exactly one appointment (the original), the new seat is free, ReschedulesCompensated increments
4. Waitlist exclusivity	Full slot, 3 waitlisted patients	Cancel one booking to free one seat	Exactly one live offer exists, the seat cannot be booked while held, expiry passes it to the next patient

13.3 Other checks

- Authentication: tampered token, expired token, wrong role and post-logout token return 401, 401, 403 and 401.
- Authorisation: a patient cannot read another patient's data, and a patient token is rejected on every doctor route.
- Every scheduled handler run twice produces no duplicate effect.
- Every refund boundary instant from Section 7.2.
- Passwords are stored as bcrypt hashes with a work factor of 12 or higher.

14. Deployment runbook

14.1 Prerequisites

An AWS account, the AWS CLI, a GitHub repository with branch protection configured, and permission to create IAM roles. Use one region for everything.

14.2 Deploy

1. Open a pull request. The pipeline lints, tests and scans.

2. After approval, merge. The pipeline packages the Lambdas, uploads them to S3 and creates and executes Change Sets in this order: network, data, auth, api, events, web, monitoring.
3. The web stack starts the EC2 instance, whose UserData downloads the Flask app from S3 and starts it.
4. The pipeline runs the integration tests and the smoke test. Open the web URL and the API base URL, register one doctor and one patient, and confirm a booking works.

14.3 Teardown

1. Empty the S3 buckets (including object versions).
2. Delete the stacks in reverse order: monitoring, web, events, api, auth, data, network.
3. Delete the SSM parameters, any restored drill resources and the bootstrap resources (OIDC provider, role, artifact bucket).
4. Check that AWS Budgets shows no remaining spend and that no resources remain in the region.

15. Submission checklist

- Every feature branch merged through an approved pull request, no open pull requests, pipeline green on master.
- No credentials anywhere in git history, including deleted commits.
- RDS reachable only from the Lambda security group. No IAM wildcard actions. All stacks IN_SYNC under drift detection.
- Architecture diagrams, this document, API documentation, test report, drill record, cost review and runbook committed.
- A submission note in the README with the web app URL and the API base URL.
- A 30-minute demo prepared: book as a patient, see it in the doctor portal, token rejections, the concurrency test, a cancellation with the right refund tier, a forced reschedule compensation, waitlist exclusivity, a pull request with its pipeline run, the dashboard with one alarm, and the architecture walked against the running system.

16. DynamoDB Schema

16.1. Refresh tokens

Attribute	**Type**	**Key / purpose**
tokenHash	String	Partition key (previously identified)
userId	String	Associated user
jti	String	Token identifier
createdAt	Number/String	Creation timestamp; confirm representation in code

expiresAt	Number	Expiration timestamp if stored in TTL separately

16.2. Slots

Attribute	Type	Key / purpose
slotId	String	Partition key
doctorId	String	Doctor associated with the slot
consultationTypeId	String	Consultation type
startTime	String	Slot start time
endTime	String	Slot end time
status	String	Slot status, e.g. OPEN/CLOSED if used
capacity	Number	Maximum capacity
bookedCount	Number	Confirmed bookings counter
heldCount	Number	Temporary holds counter
version	Number	Version for concurrency control, if implemented

16.3. Seats

Attribute	Type	Key / purpose
slotId	String	Partition key
patientId	String	Sort key
appointmentId	String	Related appointment identifier
status	String	Reservation status, if used
createdAt	String	Creation time

16.4. Waitlist

Attribute	Type	Key / purpose

slotId	String	Partition key
joinedAtPatientId	String	Sort key such as timestamp#patientId
patientId	String	Patient on the waitlist
status	String	Waitlist-entry status
createdAt	String	Entry creation time

16.5. HoldOffers

Attribute	**Type**	**Key / purpose**
offerId	String	Unique offer identifier
slotId	String	Related slot
patientId	String	Patient receiving the offer
status	String	Possible values such as LIVE/ACCEPTED/EXPIRED
createdAt	String	Creation time
expiresAt	Number	Expiration timestamp
appointmentId	String	Associated appointment, if accepted

16.6. Idempotency

Attribute	**Type**	**Key / purpose**
idempotencyKey	String	Candidate primary key
requestHash	String	Identifies request payload
status	String	Processing/completion state, if implemented
response	Map/String	Stored result, if implemented
createdAt	String	Creation time
expiresAt	Number	Expiry timestamp

17. RDS MySQL Schema

17.1. users

Column	Identified type	Key / purpose
user_id	CHAR(36)	Primary key
full_name	VARCHAR(120)	User's full name
email	VARCHAR(254)	Unique email address
password_hash	VARCHAR(255)	Password hash; never store plaintext password
role	ENUM('patient','doctor','admin')	Account role; confirm actual supported values
is_active	TINYINT(1)	Whether account is active
created_at	TIMESTAMP	Creation timestamp
updated_at	TIMESTAMP	Last update timestamp

17.2. patients

Column	Type	Key / purpose
patient_id	CHAR(36)	Primary key
user_id	CHAR(36)	References users.user_id
date_of_birth	DATE	Date of birth
gender	VARCHAR / ENUM	Gender, if stored
phone	VARCHAR	Contact number
city	VARCHAR	Patient city, if stored
emergency_contact	VARCHAR / JSON	Emergency contact details, if stored

17.3. doctors

Column	Type	Key / purpose
doctor_id	CHAR(36)	Primary key
user_id	CHAR(36)	References users.user_id

hospital_id	CHAR(36)	References hospitals.hospital_id, if applicable
specialisation_id	CHAR(36)	References specialisations.specialisation_id, if applicable
licence_number	VARCHAR	Professional licence
years_experience	INT	Experience
qualifications	TEXT / JSON	Qualifications, if stored
languages	JSON / TEXT	Languages, if stored
timezone	VARCHAR	Doctor's timezone, if stored
photo_key	VARCHAR	S3 object key, if photo upload is implemented

17.4. hospitals

Column	Type	Key / purpose
hospital_id	CHAR(36)	Primary key
name	VARCHAR	Hospital name
address	VARCHAR / TEXT	Hospital address
city	VARCHAR	City
created_at	TIMESTAMP	Creation time, if stored

17.5. specialisations

Column	Type	Key / purpose
specialisation_id	CHAR(36)	Primary key
name	VARCHAR	Specialisation name
description	TEXT	Description, if stored

17.6. consultation_types

Column	Type	Key / purpose
consultation_type_id	CHAR(36)	Primary key

doctor_id	CHAR(36)	References doctors.doctor_id
name	VARCHAR	Consultation type name
duration_minutes	INT	Consultation duration
fee_amount	DECIMAL	Consultation fee
capacity	INT	Maximum capacity, if applicable
is_active	TINYINT(1)	Whether consultation type is active

17.7. availability_patterns

Column	Type	Key / purpose
pattern_id	CHAR(36)	Primary key
doctor_id	CHAR(36)	References doctors.doctor_id
day_of_week	INT / VARCHAR	Weekday representation
start_time	TIME	Local start time
end_time	TIME	Local end time
consultation_type_id	CHAR(36)	Associated consultation type, if stored

17.8. blackout_dates

Column	Type	Key / purpose
blackout_id	CHAR(36)	Primary key
doctor_id	CHAR(36)	References doctors.doctor_id
date	DATE	Unavailable date
reason	VARCHAR / TEXT	Reason, if stored

17.9. appointments

Column	Type	Key / purpose
appointment_id	CHAR(36)	Primary key

patient_id	CHAR(36)	References patients.patient_id
doctor_id	CHAR(36)	References doctors.doctor_id
consultation_type_id	CHAR(36)	References consultation_types.consultation_type_id
slot_id	VARCHAR	Slot identifier; may refer to DynamoDB, not an RDS foreign key
status	VARCHAR / ENUM	Appointment status
fee_amount	DECIMAL	Fee recorded for appointment, if stored
created_at	TIMESTAMP	Creation time
updated_at	TIMESTAMP	Last update time

17.10. refund_ledger

Column	**Type**	**Key / purpose**
refund_id	CHAR(36)	Primary key
appointment_id	CHAR(36)	References appointments.appointment_id
refund_amount	DECIMAL	Refund amount
refund_percent	DECIMAL	Refund percentage, if stored
status	VARCHAR	Refund status
created_at	TIMESTAMP	Creation time

17.11. RDS MySQL — Entity Relationship Diagram

